

Student Information System – Project Documentation

1. Project Overview

1.1 Project Title

Student Information System (Console-Based Java Application)

1.2 Project Description

The Student Information System is a console-based Java application designed to store, manage, and display student records efficiently. It allows users to add, view, search, update, and delete student information using a menu-driven interface.

1.3 Project Objectives

- To understand and apply Object-Oriented Programming (OOP) concepts in Java
- To implement data storage using ArrayList
- To perform input validation and error handling
- To build a user-friendly, menu-driven console application
- To demonstrate clean code structure and modular design

2. System Requirements

2.1 Hardware Requirements

- Minimum 2 GB RAM
- Any standard computer system

2.2 Software Requirements

- Java Development Kit (JDK) 8 or above
- Any Java-supported IDE (IntelliJ IDEA / Eclipse / NetBeans) or Command Prompt
- Operating System: Windows / Linux / macOS

3. Setup & Installation Guide

3.1 Installation Steps

1. Install **Java JDK 8 or higher** on your system
2. Set the Java environment variable (JAVA_HOME)
3. Open your IDE or Command Prompt
4. Create a new Java project named `StudentInformationSystem`
5. Create the following files inside the project:
 - a. `Student.java`
 - b. `StudentManager.java`
 - c. `ValidationUtils.java`
 - d. `StudentInformationSystem.java`
6. Copy the provided source code into respective files

3.2 Compilation & Execution (Command Line)

```
javac *.java
java StudentInformationSystem
```

4. Code Structure & File Hierarchy

```
StudentInformationSystem/
|
|— Student.java           // Model class
|— StudentManager.java    // Business logic
|— ValidationUtils.java    // Validation methods
```

└─ StudentInformationSystem.java // Main class

5. Detailed Code Explanation

5.1 Student.java

- Represents the Student entity
- Contains attributes: studentId, name, age, grade, contact
- Uses encapsulation (private variables with getters/setters)

5.2 StudentManager.java

- Manages student records using `ArrayList<Student>`
- Implements CRUD operations:
 - Add Student
 - View Students
 - Search by ID / Name
 - Delete Student

5.3 ValidationUtils.java

- Provides reusable validation methods
- Validates:
 - Age (positive and reasonable)
 - Grade (1–12)
 - Contact number (10 digits)

5.4 StudentInformationSystem.java

- Main class containing menu-driven logic
- Uses Scanner for user input
- Handles invalid input using try-catch

6. Algorithms & Data Structures Used

6.1 Data Structure

- **ArrayList**
 - Dynamic size
 - Easy insertion and deletion

6.2 Algorithms

- **Linear Search** for searching students by ID or Name
- **Menu-driven loop** using do-while
- **Input validation logic** using conditional statements

7. Application Architecture

- **Presentation Layer:** Console-based UI (Scanner input & formatted output)
- **Business Logic Layer:** StudentManager
- **Utility Layer:** ValidationUtils
- **Data Layer:** ArrayList (in-memory storage)

8. Visual Documentation (Screenshots)

The screenshot shows an IDE window titled "Student Information System". The Explorer panel on the left shows a project structure with files: Student.class, Student.java, StudentInformationSystem.class, StudentInformationSystem.java, StudentManager.class, StudentManager.java, ValidationUtil.class, and ValidationUtil.java. The main editor displays the code for Student.java. The code defines a Student class with private attributes studentId, name, age, grade, and contact. It includes a constructor, getters, and setters for each attribute.

```
src> Student.java > Student > Student(String studentId, String name, int age, double grade, String contact) {
1 public class Student {
2     private String studentId;
3     private String name;
4     private int age;
5     private double grade;
6     private String contact;
7
8     public Student(String studentId, String name, int age, double grade, String contact) {
9         this.studentId = studentId;
10        this.name = name;
11        this.age = age;
12        this.grade = grade;
13        this.contact = contact;
14    }
15
16    // Getters & Setters
17    public String getStudentId() {
18        return studentId;
19    }
20
21    public String getName() {
22        return name;
23    }
24
25    public void setName(String name) {
26        this.name = name;
27    }
28
29    public int getAge() {
30        return age;
31    }
32
33    public void setAge(int age) {
34        this.age = age;
35    }
36
37    public double getGrade() {
38        return grade;
39    }
40
41    public void setGrade(double grade) {
42        this.grade = grade;
43    }
44
45    public String getContact() {
46        return contact;
47    }
48 }
```

The screenshot shows the same IDE window, but now displaying the code for StudentManager.java. The code defines a StudentManager class that manages a list of Student objects. It includes methods for adding students, viewing all students, and searching for students by ID or name.

```
src> StudentManager.java > StudentManager > viewStudents()
1 import java.util.ArrayList;
2
3 public class StudentManager {
4     private ArrayList<Student> students = new ArrayList<>();
5
6     // Add student
7     public void addStudent(Student student) {
8         students.add(student);
9         System.out.println("Student added successfully!");
10    }
11
12    // View all students
13    public void viewStudents() {
14        if (students.isEmpty()) {
15            System.out.println("No student records found.");
16            return;
17        }
18
19        System.out.printf(format: "%-10s %-15s %-5s %-7s %-12s\n",
20            args: "ID", "Name", "Age", "Grade", "Contact");
21        System.out.println("-----");
22
23        for (Student s : students) {
24            System.out.printf(format: "%-10s %-15s %-5s %-7s %-12s\n",
25                s.getStudentId(), s.getName(),
26                s.getAge(), s.getGrade(), s.getContact());
27        }
28    }
29
30    // Search by ID
31    public Student searchById(String id) {
32        for (Student s : students) {
33            if (s.getStudentId().equalsIgnoreCase(id)) {
34                return s;
35            }
36        }
37        return null;
38    }
39
40    // Search by name
41    public Student searchByName(String name) {
42        for (Student s : students) {
43            if (s.getName().equalsIgnoreCase(name)) {
44                return s;
45            }
46        }
47        return null;
48    }
49 }
```

```
PS C:\Users\PC\Desktop\Student Information System> javac src\*.java
PS C:\Users\PC\Desktop\Student Information System> java -cp src StudentInformationSystem

===== STUDENT INFORMATION SYSTEM =====
1. Add Student
2. View Students
3. Search Student by ID
4. Search Student by Name
5. Delete Student
6. Exit
Enter choice: 1
Enter Student ID: 101
Enter Name: sohil
Enter Age: 21
Enter Grade (1-100): 78.8
Enter Contact (10 digits): 1234567890
Student added successfully!

===== STUDENT INFORMATION SYSTEM =====
1. Add Student
2. View Students
3. Search Student by ID
4. Search Student by Name
5. Delete Student
6. Exit
Enter choice: 2
ID      Name      Age      Grade      Contact
-----
101     sohil     21       78.8       1234567890

===== STUDENT INFORMATION SYSTEM =====
1. Add Student
2. View Students
3. Search Student by ID
4. Search Student by Name
5. Delete Student
6. Exit
Enter choice: 
```

9. Testing & Validation Evidence

9.1 Sample Test Cases

Test Case	Input	Expected Output
Add Student	Age = -5	Validation Error
Add Student	Grade = 15	Invalid Grade
Search ID	Invalid ID	Student Not Found
Delete Student	Valid ID	Deleted Successfully
Contact	12345	Invalid Contact

9.2 Error Handling

- Handles non-numeric input
- Prevents invalid age, grade, and contact
- Prevents system crash using try-catch

10. Quality Standards Checklist

- ✓ Clear project objectives
- ✓ Modular and readable code
- ✓ Proper validation and error handling
- ✓ Organized file structure
- ✓ Menu-driven interface
- ✓ Test cases included
- ✓ Ready for academic submission

11. Conclusion

The Student Information System project successfully demonstrates the practical implementation of Java OOP concepts, collections, validation, and console-based application design. The project is scalable and can be extended with file handling, database integration, or GUI support in the future.

12. Future Enhancements

- File handling (save/load records)
- Database integration (MySQL)
- GUI using Swing or JavaFX
- Role-based access (Admin/User)
-

Submitted By: Mohd Sohil Khan

Technology Used: Java

Project Type: Console Application